

Automating COICOP Coding for Household Budget Surveys

ISI World Statistics Congress 2026

2026-06-06

Laura GAIMARD
laura.gaimard@insee.fr

Insee — French National Institute of Statistics and Economic Studies

01

1. Context

1.1 Some context elements

COICOP : Classification of Individual Consumption by Purpose

- Hierarchical classification : on 5 levels → **we code at level 4**
- Assigning a code requires understanding both the **product** and its **purpose**

Level	Example
1 - Division	01 - Food
4 - Sub-class	01.1.1.1 - Rice

Household Budget Survey (HBS) : new wave in 2026

- **Goal:** understand households expenditure (consumption basket)
- ~**70,000 distinct free-text labels** to code into COICOP
- New collection modes: smartphone, receipts, paper
- COICOP updated since 2017 → **limited reuse** of past labels
- Annotation team: **7 people** → **1**

Deadline: 6 months to build a production-ready pipeline

1.2 Key challenges

Classification

- ~900 granular COICOP classes (all 5 levels)

Labels

- Heterogeneous textual inputs across sources
- Generic, ambiguous, or uninformative (“*various food*”, “*article*”)
- Same string can map to different codes depending on context

Annotation

- Few annotated labels available (fewer human resources)
- Limited and unevenly distributed labelled data across COICOP divisions

Tools

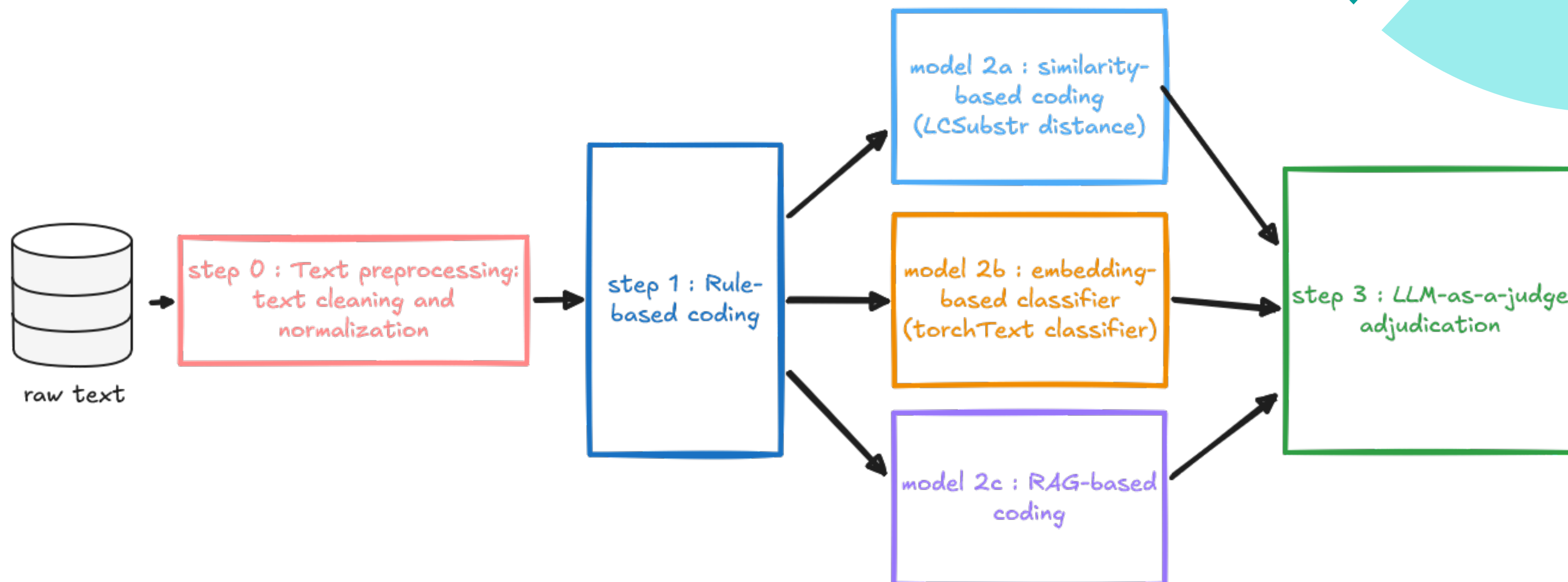
- No GPU available on production servers

02

2. The Coding Methods

THE CODING METHODS

2.1 Pipeline overview



2.2 Method 1 - LCSubstr: leveraging historical annotations

Principle

- Compares each label against a **suggester**: a curated list of reliably coded labels with their validated COICOP code
- Returns the best match and its similarity score

Example - Query: "baguette" (bread)

Suggester candidate	Code	LCSubstr distance
baguette	01.1.1.1	0.0
baguette en bois	05.5.2	0.5

→ Best match (lowest distance): 01.1.1.1

Output: top-1 candidate code + LCSubstr distance

2.3 Method 2 - TTC: embedding-based classifier

torchText Classifiers : Architecture

- Inspired by fastText
- Character n-gram tokenizer (3–6 grams) + 128-dim embedding
- **Flat** variant: direct level 4 COICOP prediction
- **Hierarchical** variant: cascade of 4 classifiers, one per COICOP level

Training data

- Pre-trained on ~**3M scanner data labels** (point-of-sale data), completed with **LLM-generated synthetic labels** for underrepresented COICOP classes
- Fine-tuned on ~**10k HBS annotations**

Output: top-k predictions + confidence scores + word attributions

2.4 Method 3 - RAG: semantic retrieval

Principle

- Embed the product label → retrieve the **top-10** most semantically similar COICOP codes from a **Qdrant** vector DB
- Inject candidates + their official COICOP notices into an LLM prompt → generate the predicted code (structured JSON)

Model choices (deployed on vLLM)

- **Embedding:** Qwen3-Embedding-8B (vector size 4096d)
- **Generation:** Gemma4-26B-MoE (temperature 0.1), structured JSON output with reflection elements

Output: predicted COICOP code (JSON) + LLM self-confidence score

THE CODING METHODS

2.5 Methods comparison

	LCSubstr	TTC	RAG
Annotation burden	Low	High	None
Process	No LLM, no GPU	GPU training / no LLM at inference	LLM + vector DB at inference
Speed	Fast (parallelised C)	Slow training / Fast at inference	Slower (LLM generation)
Quality signal	LCSubstr distance (0–1)	Calibrated confidence score	LLM self-reported confidence
Explainability	Common substring	Word attributions	LLM self-reporting
Accuracy L1	67.5%	87.6%	82.7%
Accuracy L4	47.1%	81.1%	65.5%

03

3. LLM-as-Judge

3.1 Arbitrating between methods

The arbitration step

- Receives predictions + scores from **LCS, TTC, RAG**
- The LLM sees: the product label, shop context, the three candidates, and a filtered COICOP nomenclature (codes + labels)
- Selects the most appropriate code and returns a confidence score (1–5)

Why LLM rather than majority vote?

A vote counts voices but the LLM **weights quality signals** (confidence, distance) and uses the full purchase context to resolve what no method could distinguish alone.

Two cost-saving shortcuts

1. **Consensus short-circuit** : if all three methods agree *and* TTC confidence ≥ 0.90 → **no LLM call**, prediction accepted directly
2. **Classification filtering** : only the relevant COICOP sections are injected → **4–10× fewer tokens**

3.2 Results

Pipeline coding stage	Coverage
Step 1 - Regex	
Regex pattern	8%
Manual review flagged	1%
Step 2 - Three methods	
Consensus shortcut (no LLM call)	7%
Step 3 - LLM-as-Judge	
LLM-as-Judge arbitration	84%
↳ of which: flagged for expert review	3.5%

Accuracy (~8,000 annotated labels)

– **Human baseline: ~90% at L5** (with internet access)

– **Pipeline** (no external lookup):

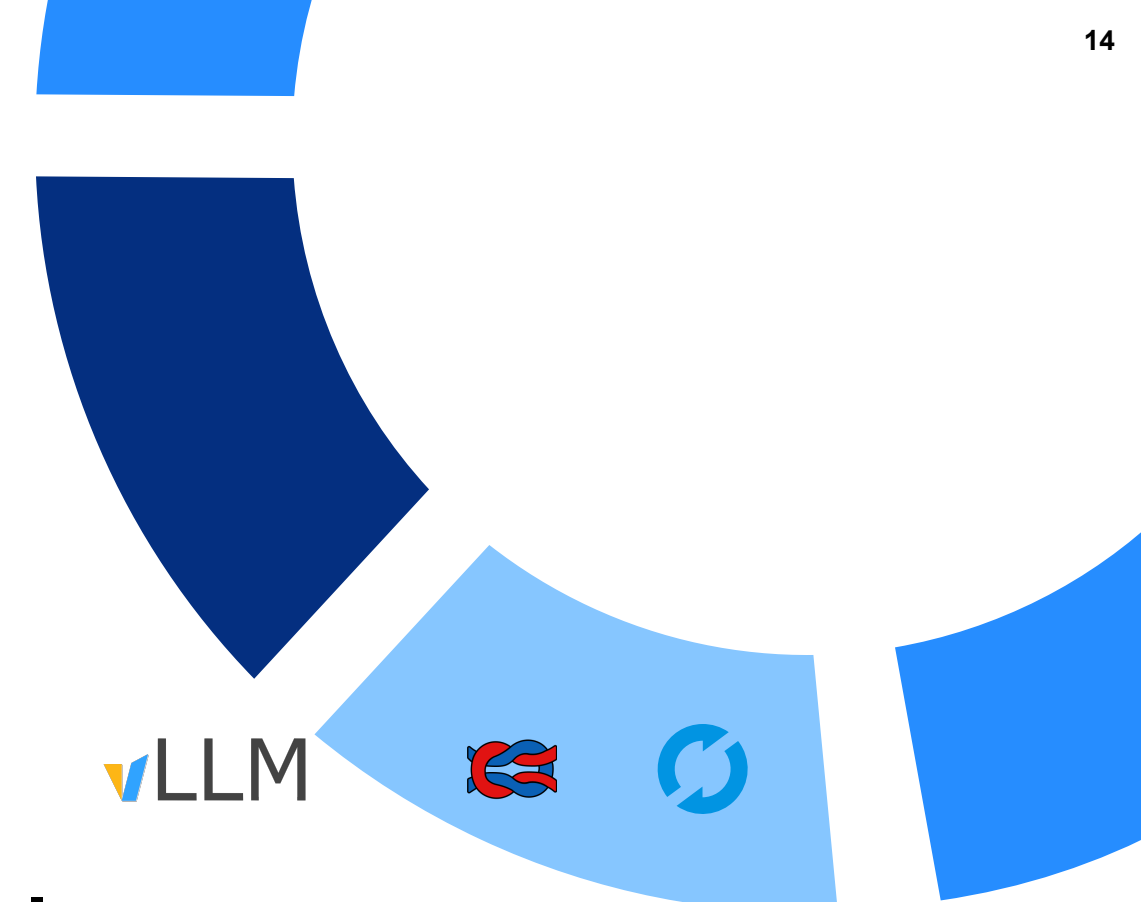
- **L1: 90.0%** → consumption baskets (best single method → TTC: 87.6%)
- **L4: 82.6%** → HBS team (best single method → TTC: 81.1%)
- **L5: 79.4%** → Eurostat (only the pipeline produces it)

Accuracy likely **underestimated**: ~10% known error rate in manual annotations (domain expert estimate).

3.3 MLOps: the experimentation stack

SSPCloud : INSEE's open data science platform, built on **Onyxia**, running on **Kubernetes**. *This platform is not a production one, used for experimentation but the pipeline is designed to be production-ready.*

- Argo Workflows sequences the 9 pipeline steps, running the three methods in parallel, then feeding results into the LLM-as-Judge
- Each step reads/writes **Parquet files on S3**
- LLMs are served via **vLLM** and exposed through an internal LLM platform providing an OpenAI-compatible API, connected to SSPCloud.



Component	Role
Argo Workflows	jobs orchestration
vLLM	LLM inference & embeddings
Qdrant	Vector database
Langfuse	Prompt versioning
MLflow	Experiment tracking

04

4. Conclusion

CONCLUSION

4.1 Limits & Improvements

Current limits

- Strong dependency on the **quantity and quality of annotated data**:
 - **Collecting 3M annotated labels** for TTC training : high effort, hard to scale
 - Accuracy **underestimated** : ~10% error rate in manual annotations inflates apparent errors
 - **Sparse coverage** for some COICOP divisions (level 1)
- Poor reliability of **confidence indicators** to target **manual review** effectively
- Deployment challenges (**GPU requirements**)

Improvements

- Add additional inputs from **2017 HBS labels**
- Improve the **LCS suggester** with richer (product, code) pair context
- Improve the **TTC model** with a **receipt-specific tokenizer**
- Log all **RAG traces** in **Langfuse**, not just prompt versioning

CONCLUSION

4.2 Conclusion

A hybrid, production-ready pipeline

- **Rule-based pre-filter** handles high-confidence, easy cases at zero ML cost
- **Three different coding methods** : LCS, TTC, RAG
- **LLM-as-Judge** arbitrates intelligently between them
- **Fully open-source MLOps** on SSPCloud

The key message

- Product and activity coding is a recurring bottleneck across official statistics.
- Our method offers a viable path forward for any survey requiring classification.
- TTC can be retrained for any classification task or language, the main requirement is an annotated training dataset.

Laura GAIMARD
Data Scientist
Directorate of statistical methodology

Insee — France

laura.gaimard@insee.fr

Insee
Mesurer pour comprendre

Thank you